VS - VUB

- **Verteiltes System**

Ein System heißt Verteiltes System, falls

→ sich Software-Komponenten auf vernetzten Computern befinden,

→ die Komponenten nur über Austausch von Nachrichten kommunizieren und

→ es dem Benutzer wie ein einzelnes kohärentes System erscheint

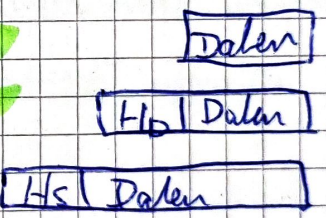
→ Internetprotokolle

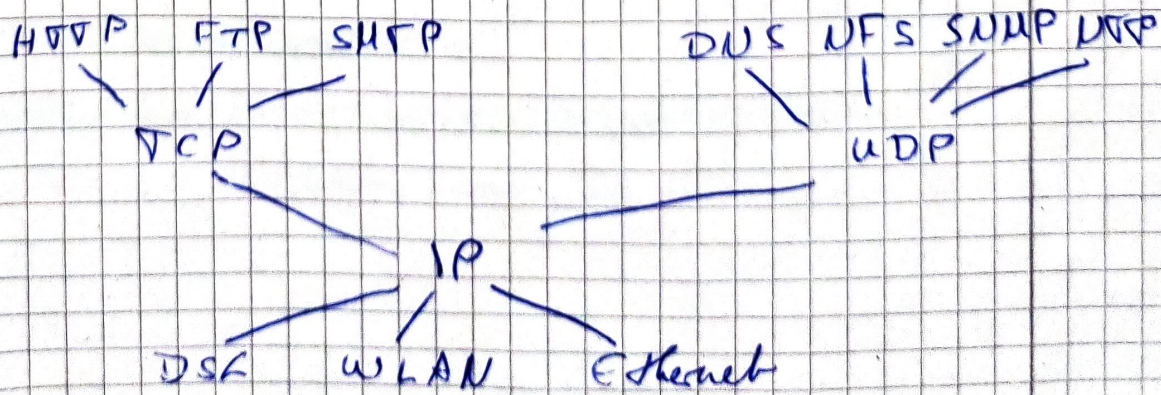
→ Internet-Kommunikation & Internetprotokolle

→ OSI Protokoll Stack

Anwendungsschicht
Darstellungsschicht
Sitzungsschicht
Transportschicht
Vermittlungsschicht
Sicherungsschicht
Bitübertragungsschicht

Jede Schicht fügt eigenen Header hinzu.





VS - Vl 05

→ Adressierung in IPv4:

→ 32 bit, 2^{32} Adressen

→ Form: 10 62 73 77 4 / 192.168.0.2 / www.foo.com

→ Umsetzung von Hostnamen in IP-Adressen durch DNS-System

→ Netz ID Rechner ID

→ Ports

→ Unter einer IP-Adresse müssen potentiell mehrere Internet-Anwendungen bedient werden

↳ Jede Anwendung wird genau ein Port-Nummer zugeordnet

→ UDP - User Datagram Protokoll

- Datagramm-orientiert, d.h. Daten als Byte-Array
  → Maximum Transmission UNIT (MTU): max Größe der Nutzlast
- Anwendung muss Daten in mehrere Datagramme aufteilen
- Leistungsmerkmale von UDP:
  → Keine Sicherheit gegen Verlust und Verfälschung
  → Keine Gewährleistung für korrekte Reihenfolge
  → Keine Bestätigung
  → Sehr geringer Verwaltungsaufwand
- UDP wird verwendet wenn
  → TCP zu aufwendig ist
  → Einzelne Pakete nicht lebenswichtig sind
  → z.B.: Internet-Telefon, Radio, DNS

VS - VL 13

→ TCP - Transmission Control Protocol

- TCP stellt Abstraktion eines Bytes-Stroms bereit, in den Daten geschrieben bzw. aus dem Daten gelesen werden.
- TCP-Verbindungen sind Voll-Duplex-Anwendungen
  → Beide Seiten können gleichzeitig senden/empfangen
- Ablauf
  → Verbindungsaufbau, Kommunikation, am Ende Verbindungsabbau
  → Nach Verbindungsaufbau kein Unterschied C/S
- Garantien
  → Erkennung doppelter Pakete
  → Reihenfolge wird eingehalten
  → Kein Verlust von Paketen.

- **Socket**

Sockets sind von BS bereitgestellte Objekte, welche als Kommunikationsendpunkte für die Interprozess-Kommunikation zwischen Prozessen auf verschiedenen Rechnern dienen.

→ Je ein Socket in den Kommunikationspartnern
→ ~~Ein~~ Socket wird an Port und IP-Adresse auf dem ausgeführten Rechner gebunden

VS - KVS

→ Datagramm Sockets (UDP)

- Serverseitig:
  - → Socket erstellen
  - → Auf Pakete warten
- Clientseitig:
  - → Socket erstellen
  - → Datagramm an Adresse senden

→ Stream Sockets (TCP)

- Serverseitig:
  - → Server-Socket erstellen
  - → Binden des Sockets an Port & IP-Adresse, über welche Anfragen angenommen werden
  - → Auf Anfragen warten ...
  - → Anfrage akzeptieren, neues Socket-Paar für diesen Client erstellen
  - → Kommunikation mit Client auf einen Client-Socket
  - → Client-Socket schließen
- Clientseitig:
  - → Socket erstellen und mit Server-Adresse, mit der kommuniziert werden soll, verbinden
  - → Kommunikation mit Server
  - → Socket schließen

VS - MUD

→ Sockets in Java: UDP

- → Klasse DatagramSocket: Datagramme senden / empfangen
  - → DatagramSocket(int port): Erzeugt Socket und bindet ihn an angegebenen Port des lokalen Rechners.
  - → void send(DatagramPacket p)
  - → void receive(DatagramPaket p)
- → Klasse DatagramPaket
  - → repräsentiert einzelnes Paket bestehend aus Bytefolge
  - → Jedes Datagramm enthält Zieladresse und Zielport.
    - → DatagramPaket(byte[] buf, int length, InetAddress address, int port)
    - → DatagramPacket(byte[] buf, int length)
  - → Methoden für Datagramme
    - → byte getData()
    - → InetAddress getAddress()
    - → int getPort()

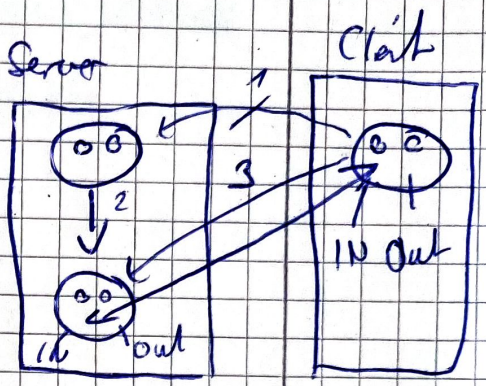
eg:

```
public static void main(String[] args) throws ... {
    DatagramSocket aSocket = new DatagramSocket(port);
    byte[] buffer = new byte[BUFFER];
    while(true){
        DatagramPacket request = new DatagramPacket(buffer, buffer.length)
        aSocket.receive(request);
        print(new String(request.getData(), 0, request.getLength())
        DatagramPacket reply = new DatagramPacket(request.getData(),
            request.getLength(), request.getAddress(),
            request.getPort());
        aSocket.send(reply);
    }
    aSocket.close();
}
```

VS - NVS

→ Sockets in Java: TCP

- Klasse ServerSocket
  - → Endpunkt einer Kommunikation, akzeptiert Anfragen und erzeugt dann Socket für eigentliche Kommunikation
  - → ServerSocket(int port)
  - → Socket accept()
- Klasse Socket
  - → Kommunikationsendpunkt für Clients
  - → Socket(InetAddress address, int port)
    Erzeugt Socket, erstellt Verbindung
  - → InputStream getInputStream()
  - → OutputStream getOutputStream()



eg:
```
public class TCP Server {
public static void main (String [] args) throws ... {
    ServerSocket serv_Soch = new ServerSocket(port);
    while (true) {
        Socket client_socket = serv_Soch.accept();
        DataInputStream in = new DataInputStream(c_Soch.getInputStream());
        DataOutputStream out = new DataOutputStream(c_Soch.getOutputStream());
        String data = in.read()
        out.write(data)
        c_Socket.close();
    }
}
```

VS - UVB

→ Persistente Datenhaltung

→ Persistente Datenhaltung durch Datenbank

→ Abstraktion: PersistentObject

→ Kapselung der Details in generischer Klasse FlatDB

→ Anwendungsprotokoll:

Anfrage-Nachricht: → Name der aufgerufenen Methode

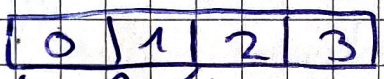
→ Parameter der Methode

Antwort-Nachricht: → Rückgabewert

→ Fehlermeldung

→ Paketstruktur:

→ Header aller Nachrichten haben gleiche Struktur



Länge des Body Typ Fehler

→ Umwandlung der Daten in Bytestrom

int → String name = decodeString(buf)

VS - NVB - Ex 1

(1-1) Wahr oder Falsch?

1) Bei TCP-Verbindungen werden alle Pakete über die selbe Route übertragen. Bei UDP können die verschiedenen Pakete über unterschiedliche Routen übertragen werden. falsch

2) TCP-Pakete sind stets größer als UDP-Pakete falsch

3) Im TCP/IP-Referenzmodell gehören HTTP & TCP zur Transportschicht falsch

4) Im OSI-Modell verdeckt die Transportschicht die Verfügbarkeit der Sitzungsschicht. falsch

5) Ein Client kann über UDP mit einem Server kommunizieren, der nur TCP-Ports geöffnet hat falsch

6) Sofern nicht anders angegeben wird TCP verwendet falsch

7) UDP-Pakete erreichen ihr Ziel schneller als TCP-Pakete falsch

8) UDP-Pakete können über IPv6 übermittelt werden richtig

9) Gemäß dem TCP-Protokoll werden empfangene IP-Pakete in der Reihenfolge an die Anwendung übergeben, in der sie in der IP-Schicht entgegengenommen wurden falsch

10) Bei der Verwendung von TCP ist zwar garantiert, dass kein Paket verloren geht, die Anwendung muss aber dennoch prüfen, ob eventuell doppelt falsch

VS – UB – Ex 1

1.2 UDP vs TCP

~~4) Virtueller Terminal-Empfänger (z.B. Telnet)~~

Szenario	Dauer	Verbindg	Performance	Protokoll
Terminal	lang	wenige	zuverlässigkeit	TCP
FTP	lang	viele	zuverlässigkeit	TCP
IP-Telefonie	lang	wenig	Kohärenz	UDP
HTTP	kurz	viele	Zuverlässigkeit	TCP (UDP)
Entfernter Prozedur	kurz	wenige	Kohärenz	UDP
DNS	kurz	viele	Kohärenz	UDP
Medien-Streaming	lang	viele	Kohärenz	UDP

Dauer der Kommunikation

kurz → UDP , lang → TCP

Anzahl der Verbindungen

viele → UDP , wenige → TCP

Verzögerungen

Meiste kurz → UDP , vernachlässigbar → TCP

Verlorene Pakete

sind verkraftbar → UDP, Daten sind auffüllt → TCP

VS - KÜB - 6.1

(1-3) Java Sockets

1) Wofür benötigt man die Methode setSoTimeout(int timeout)?

↳ Enables SO_TIMEOUT

↳ Set a timeout on blocking Socket operations.

2) Wofür dienen die Methoden setSendBufferSize(int size) und setReceiveBufferSize(int size)?

↳ Set the size of the I/O Buffers.

3) Die API bietet bereits die Methode isBound() an. Wird isConnected() dann noch gebraucht?

isBound() → returns the <u>binding state</u> of the socket also if true when socket is closed

isConnected(): returns <u>the connection state</u> of the socket.

4) Bei TCP Verbindungsaufbau ACKs versendet wird?

→ passiert automatisch

5) Welche Ports bei einer gegebenen Server für die Kommunikation mit TCP geöffnet sind?

Socket(InetAddress address, int port)

↳ IOException - if an I/O error occurs when creating the socket.

VS-UVB

(1,4) Serverimplementierung

1) Ist es sinnvoll, die Anzahl der Threads in einem Server-Prozess zu begrenzen?

   Ja, damit der Server nicht überlastet wird.

2) Server kann als Multithreaded- oder Singlethreaded-Server implementiert werden. Gibt es Szenarien, für die ein Singlethreaded-Server besser geeignet wäre?

   ↳ Single Threaded mit wenig Benutzern

   ||

   Kurze

VS - UB - 6x1

(1-8) Geschichtete Architekturen

ein mehrschichtiges Client-Server-Architekturo.
P_i ist ein Client von Prozess P_{i+1} und P_i gibt dann eine Antwort an P_{i-1} zurück, nachdem er eine Antwort von P_{i+1} erhalten hat.
Welche Probleme können bei dieser Anordnung auftreten, wenn man als Anforderung kurze Antwortzeit von Prozess P1 betrachtet?

~~P1 → P2 → P3~~

~~Text~~ ⇒ Jede Schicht auf eigenem Rechner

⇒ Von P_1 bis P_n sind n-2 Anforderungs/Antwortnachrichten nötig

⇒ Heißt durch schwächsten Rechner im Netz bestimmt

⇒ Problem beim Kopieren großer Dateien.

V5 - KVB - Praxis 1

↝ Java Thread

```
[...]
new Thread(newPlayer).start();
[...]
```

↳ starts run method of the called class

VS-KVB

=> Verteiltes System: => Software-Komponenten auf verschiedenen PCs
-> Komponenten kommunizieren nur per Nachrichten-austausch

=> Motivation für Verteilte Systeme:
=> Gemeinsame Ressourcennutzung
-> Physische Ressourcen: Festplatten, CPU
-> Logische Ressourcen: Dateien
=> Informationen austausch
=> Geographisch verteilten Zugang zu Anwendungen ermöglichen
=> Virtualisierung von IT-Systemen
=> Wirtschaftlichkeit
=> Verlässlichkeit
=> Reale Welt ist ein Verteiltes System

=> Nachteile Verteilter Systeme:
=> Neue Sicherheitsrisiken
=> Erhöhte Komplexität
=> Leistungsschwankungen

VS - VVD

- Speicherbasierte Kommunikation
  Prozesse haben Zugriff auf gemeinsamen Speicherbereich (Shared Memory). Kommunikation mittels Leser / Schreiber auf gemeinsame Variable in diesem Bereich

- Nachrichtenbasierte Kommunikation
  Kommunizierende Prozesse schicken sich gegenseitig Nachrichten über ein Netzwerk zu.

↳ Nachricht: Bytesequenz
↳ Operationen: send (Nachricht, Empfänger), receive (MSG, Sender)
↳ Voraussetzung:
  → Physikalisches Medium zwischen Kommunikationspartnern
  → Kommunikationsprotokolle, d.h. Verhaltensregeln
    ▷ Kodierungsregeln
    ▷ Nachrichtenlänge & -format
    ▷ Form der Adressierung
    ▷ Bestätigungen, Fehlercodes
  → Gemeinsame Sprache und gemeinsame Semantik

VS - NUB

→ Nachrichtenübertragung

- Ziel: Verlässliche Kommunikation
- Problem: → Kanal unzuverlässig
  → Controller können abstürzen & MSGs verlieren
- Transportschicht des Betriebssystems versucht verlässliche Kommunikation herzustellen.

- **Blockierende Funktion**
  Eine Funktion heißt blockierend, wenn sie den Programmablauf unterbricht und auf ein bestimmtes Ereignis wartet und erst bei dessen Eintritt weiterarbeitet.

- **Synchrone Kommunikation**
  Kommunikationsart bei der sendender und empfangender Prozess synchronisiert sind. Senden und Empfangen sind blockierende Operationen.

- **Asynchrone Kommunikation**
  Kommunikationsart bei der sendender und empfangender Prozess nicht synchronisiert sind. Senden ist nicht blockierend. Empfangen kann / muss aber nicht blockieren.

VS - KVB

→ Synchrone Kommunikation:

Vorteile: → Keine Mechanismen zur zeitlichen Synchronisation notwendig.
→ Keine Puffer von Nachrichten notwendig
⇒ Spart Speicherplatz und Verwaltungsaufwand

Nachteile: → Schränkt Parallelität ein
→ Ist Sender/Empfänger nicht bereit, so ist der entsprechende Empfänger/Sender dauerhaft blockiert.

→ Asynchrone Kommunikation:

- Sender blockiert nur bis send zurückkehrt:
  → send kehrt zurück, wenn MSG in Transportpuffer des Kommunikationssystems abgelegt ist. Falls Puffer voll, dann blockieren oder Sender wird über Misserfolg informiert.
  → Sender weiß nicht, ob und wann MSG gelesen wird
  → Nonresponder sendet receive testet ob MSG im Empfangspuffer liegt
  → Falls ja, so wird sie aus dem Puffer entnommen und an Anwendung übergeben. Falls nein kehrt receive zurück
- Vorteile: → zeitliche Entkopplung von Sender und Empfänger
  ⇒ höherer Grad von Parallelarbeit möglich
  → geringere Gefahr von Kommunikations-deadlocks
- Nachteile: → Puffer auf Sende- & Empfangsseite notwendig
  → Sender weiß nicht, ob und wann Puffer beim Empfänger frei ist
  ⇒ Sind Puffer voll, so muss blockiert werden
  → Sender weiß nicht, ob MSG angekommen ist.

VS - ÜB

→ Probleme verteilter Systeme:

(-) Keine globale Uhr
(-) Heterogene Teilsysteme
(-) Sicherheit
(-) Nebenläufigkeit
(-) Deadlocks
(-) Unabhängige Ausfälle

- Transparenz

Teil eines Systems der vorhanden und in Betrieb ist, aber aus Sicht unsichtbar und vom Benutzer nicht als vorhanden wahrgenommen wird.

↳ Verteilungstransparenz: Verteiltheit des Systems wird unsichtbar
↳ Ortstransparenz: Ressourcenort ist Benutzer unbekannt.
↳ Migrationstransparenz: Verbirgt Positionsveränderung von Ressourcen
↳ Replikationstransparenz: System repliziert Ressourcen und verbirgt dies vor Benutzern.
↳ Nebenläufigkeitstransparenz: Verbirgt gleichzeitige Nutzung von Ressourcen.
↳ Ausfalltransparenz: Verbirgt Ausfall und Wiederherstellung einer Ressource.

VS-MVD

- **Skalierbarkeit**

Skalierbarkeit beschreibt die Fähigkeit eines Systems, sich ändernden Ansprüchen an die Leistungsfähigkeit anzupassen.

Gut skalierbar: 1 Server 20 Benutzer ⇝ 2 Server 40 Benutzer
Schlecht skalierbar: Für doppelte Last werden bereits die zehnfachen Ressourcen benötigt.

- **Skalierungsfaktor (Speed Up)**

Der Skalierungsfaktor gibt den tatsächlichen Leistungszuwachs einer zusätzlichen Ressourcen-Einheit an.

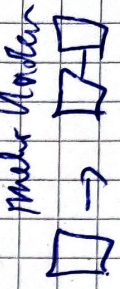
→ super-linear, linear, sub-linear

⇝ **Vertikale Skalierung:**
→ lokal vorhandene Ressourcen werden erhöht
⇒ Rechner wird mit leistungsfähigerer HW ausgestattet
→ Vorteil: In der Regel keinen Einfluss auf Software-Architektur
→ Nachteil: Vertikale Skalierung ist oft nicht linear
→ Vertikale Skalierung hat Grenzen :(

VS - VVD

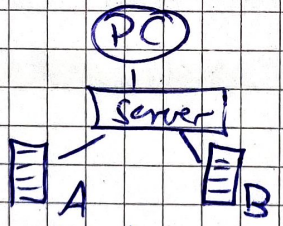
Horizontale Skalierung

- Neue Ressourcen werden über ein Netzwerk angeschlossen und Last wird auf mehrere Rechner verteilt.
- z.B.: Statt einen mehrere Server einsetzen, wobei ein Server als Lastverteiler für die anderen agiert.
- Replikation des Dienstes
  - Voraussetzung: Anwendung lässt sich replizieren
  - Vorteile: → Lastverteilung, Ausfallsicherheit
  - Nachteile: → Abhängigkeit von Netzwerkverfügbarkeit
    → Hohe Sicherheitsanforderungen

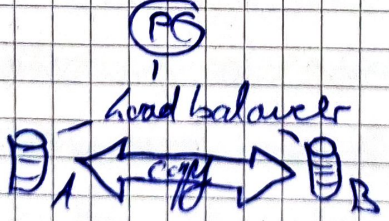


Horizontale Skalierung von Datenbanken

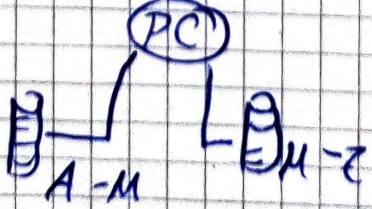
- Partitionierung: Datenbestand über mehrere Speichermedien verteilen



- Clustering: Replikation und Lastverteilung



- Sharding: Einzelne Server sind für konsistentes Datenvolumen verantwortlich



VS - RVS

→ Techniken zur Erhöhung der Skalierbarkeit:
- → Verarbeitung aufteilen und verteilen
- → Daten replizieren
- → Caching
- → Lastverteilung
- → Asynchrone Kommunikation
- → Ressourcenkonflikte minimieren
- → Zustandslose Komponenten nutzen
- → Virtualisierung

- **Elastizität**

Elastizität beschreibt die Fähigkeit eines Systems, sich dynamisch – also zur Laufzeit – an wachsende und sinkende Nachfragen anzupassen.

- **Cloud-Computing**

Beim Cloud-Computing werden IT-Ressourcen wie Speicherplatz, Rechenleistung oder Anwendungssoftware als Dienstleistung über das Internet bereitgestellt. Sie ist eine unbegrenzte horizontale Skalierung mit hoher Elastizität

↳ bietet hohen Grad an Transparenz
↳ Nutzung über technische Schnittstellen

VS-ICUB-G2

(2.0)

1) Was ist ein Verteiltes System?

Verteiltes System falls:

→ sich Softwarekomponenten auf vernetzten Computern befinden

→ die Komponenten nur über Austausch von Nachrichten kommunizieren

→ es dem Benutzer wie ein einzelnes kohärentes System vorkommt

2) Mehrprozessorsysteme vs VS?

Verteiltes Systeme	Mehrprozessorsysteme
→ Kein gemeinsamer Speicher ↳ entfernte syscalls ↳ Kommunikation über MSGs • Protokoll needed!	→ Shared Memory ↳ Direkte Kommunikation
→ Heterogenes System ↳ Middleware wird gebraucht	→ Homogenes System
→ keine Globale Uhr ↳ synchronisiert	→ Globale Uhr
→ Synchronisation von Komponenten kann zu Deadlocks führen	
→ Verhaltensfehler schwer zu kontrollieren	

VS - NVS - 6.2

(2-1)

3) Speicherbasierte & Nachrichtenbasierte Kommunikation?

Speicherbasierte Kommunikation:

Prozesse haben Zugriff auf gemeinsamen Speicherbereich
Kommunikation mittels Lesen / Schreiben auf gemeinsame Variable in diesen Bereich.

Nachrichtenbasierte Kommunikation:

Kommunizierende Prozesse schicken sich gegenseitig Nachrichten über ein Netzwerk zu.

4) Skalierbarkeit & Transparenz?

Transparenz:

Teil eines Systems der vorhanden und in Betrieb ist, aber ansonsten unsichtbar und vom Anwender nicht als existierend wahrgenommen wird.

Skalierbarkeit:

Skalierbarkeit beschreibt die Fähigkeit eines Systems, sich wechselnden Anforderungen an die Leistungsfähigkeit anzupassen.

VS-ÜB-6x2

(2.1)

5) synchrone / asynchrone Kommunikation?

Synchrone Kommunikation:

Kommunikationsart bei der sendender und empfangender Prozess synchronisiert sind. Senden und Empfangen sind blockierende Operationen.

Asynchrone Kommunikation:

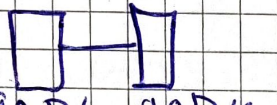
Kommunikationsart bei der sendender und empfangender Prozess nicht synchronisiert sind. Senden ist nicht blockierend. Empfangen kann, muss aber nicht blockieren.

VS-UVB - Ex 2

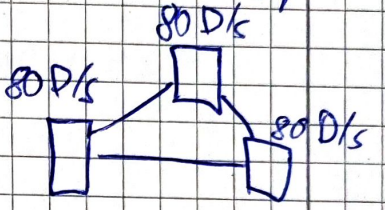
(2-2) Skalierbarkeit

Dateiserver pro Sekunde 100 Dateizugriffe von C bedienen.
→ weitere Server bekommen Kopien der Dateien.
Ein Server benötigt zur Synchronisation eine Verbindung zu jedem anderen Dateiserver.
pro Verbindung ~ 10 Dateizugriffe pro Sekunde weniger

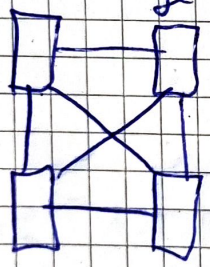
1) Wie viele Server müssen mindestens laufen, um 1 Mio. 250 Zugriffe pro Sekunde zu verarbeiten?



= 180 D/s



= 240 D/s



= 280 D/s → 4 Server!

2) Berechnen Sie die Funktion $f(n)$, die die maximale Anzahl von Dateizugriffen pro Sekunde für ein System mit n replizierten Dateiservern angibt.

~~$f(n) = n \cdot 100 - 10 \cdot (n-1) = n \cdot 100 - n \cdot 10 + 10 = 90n + 10$~~

$f(n) = n \cdot 100 - 10 \cdot (n-1) \cdot n = 100n + 10(n^2 - n)$

$= 100n - 10n^2 + 10n = 110n - 10n^2 = 10n(11 - n)$

3) Berechnen Sie ab 11 Servern nicht mehr berechtigt ✓
k ab 6 Servern fallend

VS - KVB - Ex 2

(2-3) Rätsel()

Big-Endian: Ende zuerst.
Little-Endian: Anfang zuerst.

1) A & B → Big / Little - Endianer

Welche Zahlen ergeben sich?

ω_1 = 0x00 0x24 0x1E 0x14

0 $2 \cdot 16 + 4 = 36$ $1 \cdot 16 + 14 = 30$ $1 \cdot 16 + 4 = 20$

ω_2 = 0x01 0x02 0x03 0x04

1 2 3 4

ω_3 = 0x05 0x06 0x06 0x05

5 6 6 5

2)

ω_1: $A = 0 \cdot 2^{24} + 36 \cdot 2^{16} + 30 \cdot 2^{8} + 20 \cdot 2^{0} = 2\,366\,996$

$B = 0 \cdot 2^{0} + 36 \cdot 2^{8} + 30 \cdot 2^{16} + 20 \cdot 2^{24} = 337\,579\,616$

ω_2: $A = 1 \cdot 2^{24} + 2 \cdot 2^{16} + 3 \cdot 2^{8} + 4 \cdot 2^{0}$

$B = 1 \cdot 2^{0} + 2 \cdot 2^{8} + 3 \cdot 2^{16} + 4 \cdot 2^{24}$

ω_3: $A = B = 5 \cdot 2^{24} + 6 \cdot 2^{16} + 6 \cdot 2^{8} + 5 \cdot 2^{0}$

First byteweise decimal

Dann A: $a_1 \cdot 2^{0} + a_2 \cdot 2^{8} + \ldots$

B: $a_1 \cdot 2^{24} + a_2 \cdot 2^{16} + \ldots$

VS-KVB-Ex2

(2-B) htonl()

2) htonl() = converts the unsigned integer host long from host byte order to network byte order

VS-RVB-6x2

(2-4) Zwei Phasen Commit Protokoll

Als Transaktion wird eine Gruppe von logisch zusgd Datenoperationen bezeichnet. → Entweder vollständig oder gar nicht.
Alle Aktionen werden rückgängig gemacht, wenn Fehler auftritt:
↳ Kein Fehler commit.

2PC: verteilte Transaktion wird von dem Koordinator durchgeführt.

Erste Phase → Zustimmung / Ablehnung der Transaktion aller beteiligten Rechner

Commit ← wenn alle Zustimmen → abort

Phase 2: → Übermittlg des Ergebnisses an alle Teilnehmer
→ Teilnehmer führen ein commit oder abort durch
→ Danach wird der Koordinator informiert.

1) Beispiel für real existierende verteilte Transaktion?

Durchführung einer Überweisung auf dem Datenbanksystem der Bank des Überweisers und dem Datenbanksystem auf des Empfängers.
Geht nach Geldtransfer auf den zweiten Bankkonto schief muss das Geld zurück gebucht werden.

VS - Übung 2

(2-a)

2) Wir haben nach aber zustand zurückgesetzt wurde.

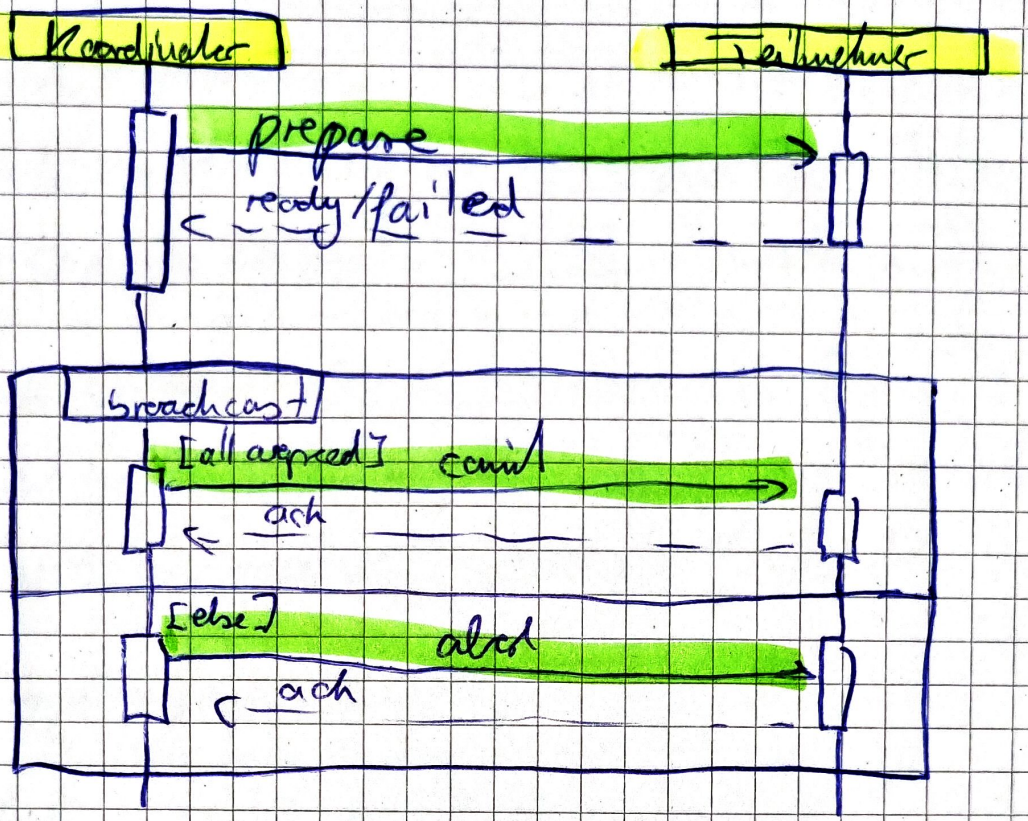
↳ Teilnehmer schauen in den Log und sehen so die Variable zurück

VS - Systeme - 6 < 2

2-4

3)

a)



4) a) Wann ist dieser Ausschnitt von Nein Fehler aufgetreten?

alle Agreed => commit Koordinator (=) bei lokales commit

b) Wie viele MSGs bei n T°?

2n + 2n = 4n MSGs

VS-Ü 03 Ex2

(2-1)

5)

a) Abstrakte Koordinator.

Stürzt der Koordinator vor dem Empfangen aller Antworten ab, so sind die Teilnehmer blockiert und werden. :(

b) Abstrakte Teilnehmer

Stürzt ein Teilnehmer ab, d.h. empfängt der Koordinator keine Antwort auf prepare, so ist der Koordinator blockiert.

c) vorangegangene Nachricht.

Blockiert Koordinator &/or Teilnehmer

VS - RVB - Prise 2

→ C++ Sockets:

CMake

For win32 → winsock2 → target_link_libraries(target
wsock32
ws2_32)

else → sys/socket.h
→ arpa/inet.h
→ winstd.h

↳ Cause (Win32):

↳ WSADATA structure: contains infos about the Windows
socket implementations.

↳ WSA Startup: Initiates use of the Winsock DLL by
a process

↳ socket: creates a socket that is bound to a specific
transport service provider

↳ socket(af, type, protocol)

↳ ~~conn~~ connect: confers socket to remote host